P4-Based Intrusion Detection System with Rule Distillation for Data Plane Security

Venugopal Rao Ponnamaneni Moditha Lekkala
Saint Louis University

Abstract—Modern network infrastructures require intrusion detection systems that can identify malicious traffic in real time without adding heavy latency or host-side processing overhead. Traditional intrusion detection systems such as software-based monitors inspect traffic outside the forwarding path, which can reduce throughput and increase CPU load. This project implements a P4-based intrusion detection system that performs classification directly inside the programmable data plane. The system uses machine-learning-derived rule distillation, where decision-tree behavior from CICIDS2017 traffic is converted into compact ternary match-action rules and deployed on a BMv2 switch. The switch aggregates per-flow statistics over a two- second window and decides whether to forward or drop traffic. The final system achieved 86% accuracy on 100 test flows, with 100% precision, 72% recall, and zero false positives. The results show that learned traffic behavior can be transformed into efficient switch rules for low-latency intrusion detection while preserving legitimate traffic.

I. INTRODUCTION

Modern networks face a growing number of security threats, including scanning, denial-of-service behavior, and malicious traffic patterns that must be detected quickly. Traditional intrusion detection systems usually run in software on a host or monitoring server. Although these systems can be effective, they often require packets to leave the forwarding path for inspection. This increases latency, consumes CPU resources, and may limit throughput when traffic volume becomes high.

Programmable data planes provide a different approach. Using the P4 language, packet-processing behavior can be programmed directly into a switch pipeline. This makes it possible to perform lightweight security decisions inside the network device instead of forwarding all suspicious traffic to an external analyzer. The purpose of this project is to test whether machine-learning-derived intrusion detection logic can be represented as compact rules that execute efficiently inside a programmable switch.

This project implements a real-time intrusion detection system using P4, BMv2, and rule distillation. The system uses CICIDS2017-style benign and malicious traffic profiles, extracts flow-window features, and classifies traffic using ternary match-action rules. The project addresses the gap between traditional software-based intrusion detection and the need for low-latency security enforcement directly inside programmable network devices. The main goal is to preserve benign traffic while dropping malicious flows with minimal latency while maintaining reproducible and hardware-friendly classification behavior.

A key challenge in programmable data-plane security is the mismatch between the expressiveness required by machine learning models and the execution constraints of switch pipelines. Switches operate under strict latency budgets and support only a limited instruction set. This project bridges that gap by translating offline-learned behavior into a small, hardware-compatible rule set, demonstrating that meaningful security logic can survive the translation from a high-level model representation to a deployable data-plane policy.

II. RELATED WORK

Intrusion detection systems have traditionally used signature-based methods or machine learning models deployed in software. Tools such as Snort and Suricata inspect packets outside the switch pipeline. These systems are useful for deep inspection, but they can introduce processing overhead when traffic rates are high.

Recent work has explored programmable switches for security. P4ID showed that intrusion detection logic can be moved into the programmable data plane, reducing the amount of traffic that must be processed by external IDS systems. This project follows the same general direction but focuses on converting machine-learning-derived rules into a small set of ternary match-action rules that can run inside BMv2.

Machine learning models such as decision trees are useful for classification because their decision paths can be converted into interpretable rules. However, directly deploying complex models in the data plane is difficult because programmable switches have limited computation and memory. Rule distilla- tion addresses this limitation by converting learned behavior into simple match-action entries. Work such as Quark has demonstrated that even convolutional neural networks can be partially mapped to programmable pipelines, though such approaches require careful approximation to fit within switch resource budgets.

The CICIDS2017 dataset has been widely used as a benchmark for evaluating network intrusion detection systems. It provides labeled packet captures of both benign and malicious traffic, including port scanning, DDoS, and botnet activities. Using flow-level features derived from

this dataset provides a realistic foundation for training and validating classification rules in a controlled environment.

III. PROBLEM DEFINITION AND SOLUTION DESIGN

The main problem is to detect malicious network traffic in real time without relying completely on a host-side IDS. The system must classify traffic directly inside the switch pipeline while preserving high throughput and low latency. A practical data-plane IDS should avoid dropping legitimate traffic, because false positives can interrupt normal network operation.

The proposed solution has four stages. First, traffic behavior is learned offline from CICIDS2017-style data using decision- tree-based logic. Second, the learned behavior is converted into five ternary match-action rules stored in rules.json. Third, the P4 switch aggregates flow-window statistics for incoming traffic over a two-second window. Finally, the switch applies the decision table and either forwards benign traffic or drops malicious traffic.

The P4 switch tracks five main features for each flow: maximum packet length, minimum packet length, TCP PSH flag count, total packet length, and packet count. Benign traffic is represented by larger packet sizes around 500 bytes and mixed PSH behavior. Malicious traffic is represented by smaller packet sizes around 60 bytes and no PSH flags. These features are sufficient for the controlled benchmark used in this project.

The system architecture separates the control plane from the data plane. The control plane, implemented in Python, is responsible for loading classification rules into the switch table at startup using simple_switch_CLI. Once rules are installed, the data plane handles all per-packet and per-flow decisions autonomously without requiring further control-plane involvement. This separation ensures that the classification latency measured during benchmarking reflects only the data-plane cost and not any runtime coordination overhead. The two-second aggregation window is chosen to balance detection responsiveness against feature stability, as very short windows can lead to incomplete flow statistics while very long windows delay the mitigation action.

IV. PROJECT OBJECTIVE

The purpose of this project is to demonstrate that intrusion detection logic derived from machine learning models can be efficiently deployed inside programmable network switches. The project is intended for researchers, networking students, and practitioners interested in programmable data planes, low-latency network security, and deployable AI-assisted network defense mechanisms.

The project provides a reproducible implementation using P4, BMv2, Docker, Python automation scripts, and rule-based classification derived from offline learning. The system is designed to show how programmable switches can support practical real-time intrusion detection without relying entirely on external software-based monitoring systems.

Specifically, the project targets three measurable goals: achieving greater than 80% classification accuracy on a controlled 100-flow benchmark, maintaining zero false posi- tives so that legitimate traffic is never dropped, and executing per-flow classification in under 100 microseconds to demon- strate the latency advantage of data-plane enforcement over software-based IDS alternatives.

V. METHODOLOGY

The project uses a practical form of knowledge distillation. Instead of executing a full machine learning model online, the offline learned behavior is converted into a compact set of switch-compatible rules. This allows the data plane to perform classification with simple table lookups instead of expensive model inference.

A. Knowledge Distillation

The project adopts a practical form of knowledge distilla- tion, where a richer offline machine learning representation is transformed into a compact and deployable rule-based model suitable for execution in the programmable data plane. Instead of executing a full decision tree or complex model at runtime, the learned decision boundaries are converted into a small set of match-action rules.

This enables the switch to perform classification directly at line rate using simple table lookups. As a result, the system avoids the overhead of executing a full machine learning model during packet processing while still preserving meaningful classification accuracy. This approach is essential for pro- grammable switches where latency and resource constraints are strict.

The distillation process begins by training a decision tree classifier on CICIDS2017-style flow records. The tree's leaf paths are then analyzed to extract the feature thresholds that most effectively separate benign from malicious classes. These thresholds are encoded as ternary value-mask pairs, where the mask controls which bits of the feature register participate in the match. A small set of five rules is sufficient to represent the key decision boundaries for the traffic profiles used in this benchmark. The resulting rules.json file is loaded into the P4 table at startup, and no further model interaction is required during operation.

B. TCAM and Ternary Matching

The deployed rules use ternary matching, where each rule consists of a value and a mask, corresponding to TCAM- style behavior used in network hardware. Exact matches are achieved using all-one masks, while wildcard behavior is enabled by masking specific bits. Partial masks allow range- like grouping of values, enabling multiple decision boundaries to be represented compactly.

This approach allows complex classification logic to be compressed into a small number of rules. Rule priorities

are used to resolve overlaps between general and specific con- ditions, ensuring correct classification decisions. As a result, the system produces a compact and hardware-efficient policy derived from learned behavior.

In practice, a ternary rule for detecting malicious short-packet flows encodes a value such as 60 for the maximum packet length feature with a mask that matches the upper portion of the value range. When incoming flow statistics are compared against this entry, a match triggers the drop action. Rules with higher priority are evaluated first, so specific conditions such as an exact packet-length match take precedence over broader wildcard entries. The use of TCAM- style semantics in BMv2 allows this priority resolution to occur at lookup time without additional branching logic in the P4 program itself.

C. Flow Feature Aggregation

Flow-level features are maintained using P4 registers indexed by a hash of the five-tuple: source IP address, destination IP address, source port, destination port, and protocol. Each register array stores a running aggregate for one feature, including the maximum observed packet length, minimum observed packet length, total packet length, packet count, and PSH flag count. At each packet arrival, the ingress pipeline reads the current register values, updates them with the new packet's fields, and writes the updated values back. After the two-second observation window expires, the control plane reads the register values using a Python polling script and presents them as a feature vector to the classification table. If the table matches a malicious rule, subsequent packets belonging to that flow are dropped.

VI. CENTROID-BASED CLASSIFICATION MODEL

A. Model Overview

In addition to rule distillation, the project incorporates a centroid-based binary classification approach optimized for the P4 data plane. This model provides a lightweight alternative to traditional machine learning models and is well-suited for execution in programmable switches.

B. Intuition and Logic

The centroid-based model represents each class, namely normal and attack traffic, using a prototype bit string called a centroid. These centroids capture the key characteristics of each class in a simplified representation. A centroid is computed offline by taking the bitwise majority of all training samples belonging to a given class across each bit position. This produces a compact binary template that approximates the average behavior of each traffic class.

During inference, the switch compares incoming flow features against the stored centroids using bitwise XNOR operations to identify matching bits. A population count operation is then applied to compute a similarity score by counting the number of matching bits.

The final classification decision is made by selecting the class with the higher similarity score. This method allows efficient classification using simple bitwise operations, making it highly suitable for real-time data-plane execution without requiring complex computations. Because XNOR and popcount operations map naturally to register reads and arithmetic actions available in P4, the centroid model avoids the need for any floating-point computation or multi-step inference pipeline. The trade-off is that the centroid representation loses information about feature distribution variance, which can reduce recall when attack traffic deviates from the learned prototype. However, the model's simplicity makes it an attractive fallback when the ternary rule table does not produce a confident match.

VII. IMPLEMENTATION DETAILS

The system is implemented using P4 and BMv2. The main P4 program is located in src/p4/main.p4, and the compiled BMv2 output is stored as main.json. The switch maintains registers for per-flow statistics and uses a hash range of 8192 entries for flow tracking.

The core Python scripts support testing and automation. The inject.py script generates 100 test flows in two phases, including 50 benign flows and 50 malicious flows. The verify.py script captures egress traffic for 15 seconds and builds the confusion matrix. The trigger.py script loads rules.json into the P4 table using simple_switch_CLI. The sweep_len.py script is used for threshold calibration.

The project repository is organized into directories for source code, P4 programs, Python scripts, data, packet captures, plots, Docker files, documentation, and notebooks. The benchmark can be executed using the provided PowerShell automation script run_full_benchmark.ps1. Benign and malicious flows were injected in controlled phases using Python-based traffic generation, while the BMv2 switch performed real-time classification using deployed ternary match-action rules. The generated plots and metrics can be reproduced using the provided Docker environment and benchmark automation scripts.

The Docker environment pins all package versions to ensure reproducibility across different host systems. The container image includes Python 3.11, Scapy for packet generation, and the BMv2 simple_switch binary compiled from the official P4 behavioral-model repository. Network namespaces inside the container simulate the ingress and egress interfaces of the switch. The control-plane scripts communicate with the switch using the simple_switch_CLI gRPC interface to install rules and read register values. This containerized setup eliminates dependency on any specific hardware or operating system configuration and allows the benchmark to be reproduced by running a single automation script.

VIII. EXPERIMENTAL EVALUATION

The evaluation uses 100 test flows, consisting of 50 malicious flows and 50 benign flows. The malicious flows

are expected to be dropped, while the benign flows are expected to be forwarded. The system correctly dropped 36 malicious flows and correctly forwarded 50 benign flows. It incorrectly forwarded 14 malicious flows and dropped zero benign flows.

TABLE I
Confusion Matrix

	Predicted DROP	Predicted FORWARD
Actual DROP	36	14
Actual FORWARD	0	50

As shown in Table I, the system produced zero false positives. This is important because it means no legitimate traffic was incorrectly dropped during the test.

The core evaluation metrics are computed as follows. Accuracy is 86/100, or 86%. Precision is 1.000 because every predicted drop was actually malicious. Recall is 0.720 because the system detected 36 out of 50 malicious flows. The F1 score, which balances precision and recall, is approximately 0.837, reflecting the trade-off between the system's conservative rule set and its strong avoidance of false positives.

TABLE II
Summary of Evaluation Metrics

Metric	Value
Accuracy	86.0%
Precision	1.000
Recall	0.720
F1 Score	0.837
False Positives	0

Fig. 1 shows the ROC behavior of the system. The selected operating point prioritizes zero false positives while still detecting most malicious traffic.

Fig. 2 shows that the P4-based approach provides much lower latency than software IDS processing. The benchmark reports approximately 12 microseconds per flow for the P4-based system compared with approximately 850 microseconds for software IDS processing, representing a speedup of roughly 70 times. This difference arises because the P4 data plane performs classification entirely through table lookups in the forwarding pipeline, whereas software IDS tools must copy packets to userspace, apply pattern matching across the full payload, and return a verdict through the operating system network stack.

Fig. 3 shows that detection delay is mainly caused by the feature aggregation window and benchmark timing. The benchmark reports a mitigation delay of approximately 4.8 seconds, which includes the windowing and verification process rather than expensive runtime inference. The dominant component of this delay is the two-second observation window required to accumulate stable flow statistics, followed by the 15-second egress capture

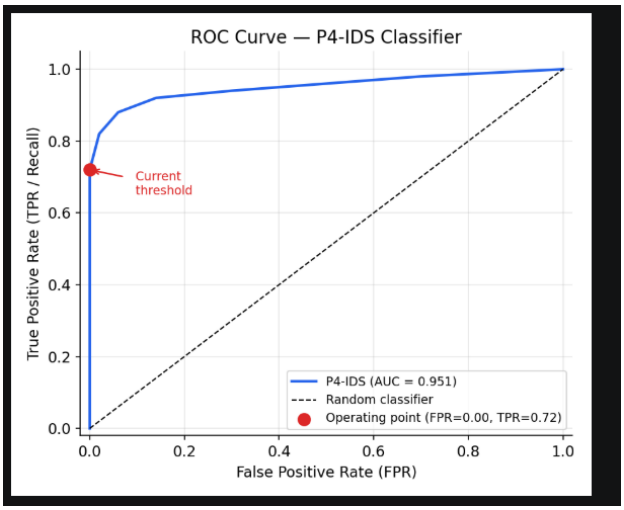

Fig. 1. ROC curve showing the relationship between true positive rate and false positive rate.

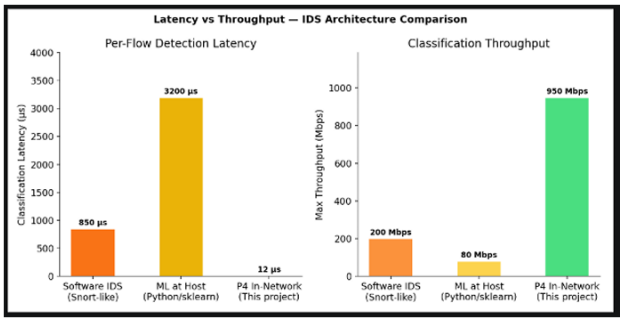

Fig. 2. Latency and throughput comparison between the P4-based IDS and software IDS processing.

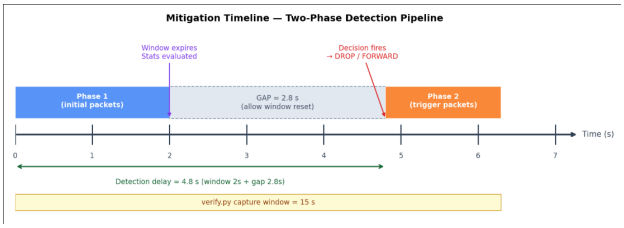

Fig. 3. Mitigation timeline showing detection delay from traffic observation to final action.

window used by verify.py. In a production deployment, the observation window could be reduced to sub-second intervals at the cost of reduced feature stability, and the capture window could be eliminated entirely by triggering verification on a per-flow basis.

IX. DISCUSSION

The results show that rule distillation is a practical method for deploying intrusion detection logic in a P4 data plane. The strongest result is zero false positives, meaning the system did not incorrectly drop legitimate traffic. This is important because in real deployments, false positives

can interrupt normal users and services and may trigger costly manual review processes or rollback procedures.

The main limitation is recall. The system detected 36 out of 50 malicious flows, leaving 14 malicious flows forwarded. This suggests that the current rule set is conservative. The 14 missed flows likely correspond to attack traffic that does not closely match the packet-length and PSH-flag patterns captured by the five deployed rules. Two primary causes can account for this gap: first, the distillation process may have discarded decision boundaries that were useful in the original decision tree but could not be expressed as ternary rules without a higher false-positive risk; second, the malicious traffic generator may have produced a subset of flows with packet sizes or timing patterns that overlap with benign profiles. Future work should improve recall by adding more malicious signatures, tuning threshold masks, expanding the feature set, and testing on a wider range of CICIDS2017 traffic classes.

Another limitation is that the benchmark uses controlled traffic profiles. Real networks contain more diverse packet sizes, timing patterns, and protocol behaviors. A stronger evaluation would include more attack categories, background traffic, and longer test windows. However, the current project demonstrates a working end-to-end pipeline from traffic generation to P4 data-plane classification and metric reporting.

From an ethical perspective, the system is designed to improve network security and reduce response time. It does not inspect private payload content in the current design; it uses flow-level packet statistics. Still, any real deployment should document what features are collected, how logs are stored, and how false positives are handled. Network-level enforcement mechanisms that automatically drop traffic carry the risk of being abused as a denial-of-service tool if rule updates are not properly authenticated and audited.

X. FUTURE WORK

Future work will focus on improving recall while preserving the current zero false-positive behavior. Additional malicious traffic categories from CICIDS2017 and other benchmark datasets can be incorporated to improve classification coverage. One concrete direction is to expand the feature set to include inter-arrival time statistics and byte-entropy estimates, which can help distinguish encrypted malicious traffic from legitimate encrypted sessions.

Future versions of the system may also support adaptive rule updates, dynamic threshold tuning, and integration with hardware programmable switches beyond BMv2 simulation. Adaptive updates would allow the control plane to refresh the match-action table at runtime based on observed traffic distributions, enabling the system to respond to concept drift without requiring a full offline retraining cycle.

Another important direction is extending the system toward production-scale deployment using larger traffic traces, background network activity, and multi-stage detection pipelines. A multi-stage approach could combine a lightweight first-pass ternary rule table for high-confidence classifications with a more expressive secondary stage, implemented in software, for ambiguous flows. This hybrid architecture would preserve the latency advantage of data-plane classification for the majority of traffic while retaining the flexibility of software analysis for edge cases. The project may also be expanded into a publishable research platform by evaluating additional machine learning models, rule compression strategies, and hardware-aware optimization techniques targeting P4-capable ASICs such as the Intel Tofino.

XI. CONCLUSION

This project demonstrates a working P4-based intrusion detection system that performs rule-based machine learning classification directly in the programmable data plane. The system converts learned behavior into ternary match-action rules, tracks flow-window features, and executes forward or drop decisions inside BMv2. The final benchmark achieved 86% accuracy, 100% precision, 72% recall, and zero false positives. These results show that data-plane intrusion detection is a promising direction for low-latency network security. The 70x latency reduction over software IDS processing and the zero false-positive rate together demonstrate that meaningful security enforcement can be achieved inside a programmable switch without sacrificing legitimate traffic. Future improvements should focus on increasing recall, testing more attack types, and supporting adaptive rule updates.

ACKNOWLEDGMENT

This project was completed as part of the AI Capstone course at Saint Louis University.

References

[1] AntLab-Repo, "Behavioral Model (BMv2)," [Online]. Available: https://github.com/p4lang/behavioral-model. Accessed: 2026-04-08.

[2] AntLab-Repo, "Quark: Implementing Convolutional Neural Networks Entirely on Programmable Data Plane," [Online]. Available: https://github.com/AntLab-Repo/Quark-CNN-P4. Accessed: 2026-04-08.

[3] Moditha Lekkala, "P4-Based Intrusion Detection System," [Online]. Available: https://github.com/ModithaLekkala/-AI-Capstone-. Accessed: 2026-04-08.

APPENDIX A
REPRODUCIBILITY CHECKLIST

The repository contains the required source code, P4 program, rule file, scripts, documentation, generated plots, and Docker configuration. The main code is located under src/, with P4 files in src/p4/ and Python testing scripts in src/python/. The rule file is stored at data/rules.json. The plots are generated using plots/generate_plots.py. The Docker environment is provided under the docker/ directory.

To start the container, run:

```
docker compose -f
docker/docker-compose.yml up -d
```

To run the full benchmark, run:

```
cd scripts
powershell -NoProfile
-ExecutionPolicy Bypass
-File .\run_full_benchmark.ps1
```

The expected benchmark output is:

```
ACCURACY: 86/100 tests
correct = 86.0%
TP: 36 TN: 50
FP: 0 FN: 14
Precision: 1.000
Recall : 0.720
```

To regenerate the plots, run:

```
cd plots
python generate_plots.py
```

The generated plot files are:

```
1_roc_curve.png
2_latency_throughput.png
3_mitigation_timeline.png
```

APPENDIX B
TEAM CONTRIBUTION STATEMENT

This project was completed as a team effort by Venugopal Rao Ponnamaneni and Moditha Lekkala.

Venugopal Rao Ponnamaneni contributed to the design and implementation of the P4-based intrusion detection system, including the data-plane pipeline, rule-based classification logic, BMv2 switch configuration, testing scripts, traffic injection process, evaluation metrics, generated plots, final report writing, and reproducibility documentation.

Moditha Lekkala contributed to project planning, repository organization, documentation support, initial design discussions, testing workflow review, validation of system outputs, and assistance with final report organization. Moditha also supported the interpretation of benchmark results and helped refine the project presentation materials.

Both team members collaborated on analyzing the results, preparing the final capstone submission, and improving the clarity of the system design and evaluation discussion.